

Introduction à la programmation



Katy SAINTIN
Septembre 2016



Ingénieur logiciel depuis 18 ans

✓ *Apprentie ingénieure au*



✓ *SSII ALTEN*



✓ *SOLEIL*
SYNCHROTRON



Mais formatrice débutante 😊 !

✓ *Partage des connaissances*

✓ *Faire découvrir le métier de développeur*

✓ *Méthodologie concrète et objective*

✓ *Toutes questions sont les bienvenues !*

Qui a déjà un expérience en développement ?

- ✓ *Professionnellement ?*
- ✓ *Autodidacte ?*

Si oui en quels langages ?

- ✓ *Java*
- ✓ *C++*
- ✓ *Site web dynamique (php, javascript)*
- ✓ *Applications mobiles*
- ✓ *Autres*

Historique

- ✓ *Origine*
- ✓ *Java rencontre Internet*
- ✓ *Philosophie*
- ✓ *Où trouve-t-on Java ?*

Environnements de développement intégrés (IDE)

- ✓ *NetBeans*
- ✓ *Eclipse*
- ✓ *Android Studio*

1990 : Patrick Naughton de Sun Microsystems.

Limitations du langage C++

- ✓ *Interface de programmation*
- ✓ *Héritage multiple*
- ✓ *Constructeur, destructeur, gestion mémoire*
- ✓ *Les pointeurs et les références.*

Limitations des outils associés.

- ✓ *Les erreurs non détectées avant compilation*
- ✓ *Introspection des classes, inexistante*

1996 : James Gosling, Mike Sheridan et Bill Joy

- ✓ *Système de ramasse-miette (garbage collector)*
- ✓ *Plate-forme portable sur tout type d'appareils ou d'OS.*
- ✓ *Nouveau langage objet Oak (chêne)*
- ✓ *1994 : adaptation à la plate-forme web (Mosaic)*
- ✓ *1994 : Oak devient JAVA*
 - *James Gosling, Arthur Van Hoff et Andy Bechtolsheim*
 - *Ou Just Another Vague Acronym*
- ✓ *9 janvier 1996 : Lancement public dans Netscape*
- ✓ *2009 : Oracle rachète Sun Microsystems.*

Historique : les versions de java

Version	Année de livraison	Dénomination	Nom de code	Spécifications
1.0	23 janvier 1996	Java 1.0	Oak	Version initiale 211 classes et interfaces
1.1	19 février 1997	Java 1.1	Sparkler	+ AWT ¹ , JavaBeans, JDBC ² , RMI ³ 477 classes et interfaces
1.2	9 décembre 2000	J2SE 1.2	Playground	API réflexion, Swing, CORBA, Collections 1 524 classes et interfaces
1.3	8 mai 2000	J2SE 1.3	Kestrel	HotSpot JVM, JavaSound (audio), JNDI ⁴ , JPDA ⁵ 1 840 classes et interfaces
1.4	6 février 2002	J2SE 1.4	Merlin	API Logging, Image I/O, XML, JCE ⁶ , Java Web Start 2 723 classes et interfaces
1.5	30 septembre 2004	J2SE 5.0	Tiger	Générique, Annotation, Autoboxing, Enumération, for 3 270 classes et interfaces
1.6	11 décembre 2006	Java SE 6	Mustang	Améliorations XML, JDBC, Java Web Start, Collection, IO 3 777 classes et interfaces
1.7	7 juillet 2011	Java SE 7	Dolphin	Switch (String), opérateur Diamant, Multicatch, Task, UI ⁷ 8000 classes et interfaces
1.8	15 mars 2014	Java SE 8	Kenaï	Expressions lambdas, Java FX > 8000 en cours de développement
1.9	? 2017	?	?	?

¹ **AWT** = Abstract Window Toolkit

² **JDBC** = Java DataBase Connectivity

³ **RMI** = Remote Methode Invocation

⁴ **JNDI** = Java Naming and Directory Interface (Annuaire LDAP)

⁵ **JPDA** = Java Platform Debugger Architecture

⁶ **JCE** = Java Cryptography Extension (Sécurité)

⁷ **UI** = User Interface (transparence des Frames, bords arrondis, Look and Feel)

Orienté objet

- ✓ *Souple, modulaire et évolutif.*

Portable

- ✓ *Indépendant de la plateforme grâce à la JVM.*

Open source

- ✓ *Nombreuses contributions sur internet*

Simple

- ✓ *API (Application Programming Interface) complète*
- ✓ *Patron de conception intégré au langage*



Minecraft

- ✓ *Jeu de construction collaboratif et créatif de gestion de ressources.*



Android

- ✓ *Applications mobiles avec API Google*



Frozen Bubble

- ✓ *Jeu de puzzle tel que Tetris*



Environnements de développement intégrés (IDE)

- ✓ *pour augmenter la productivité des programmeurs*
- ✓ *éditeur de texte*
- ✓ *compilateur*
- ✓ *débogueur*



<http://netbeans.org/>



<http://eclipse.org/>



<http://developer.android.com>

Chapitre 2 : Structure d'un programme JAVA

Environnement d'exécution d'un programme

- ✓ *La compilation*
- ✓ *Objet, classe, pointeur et référence*
- ✓ *L'utilisation des packages*
- ✓ *Le CLASSPATH*

Les bonnes pratiques en développement

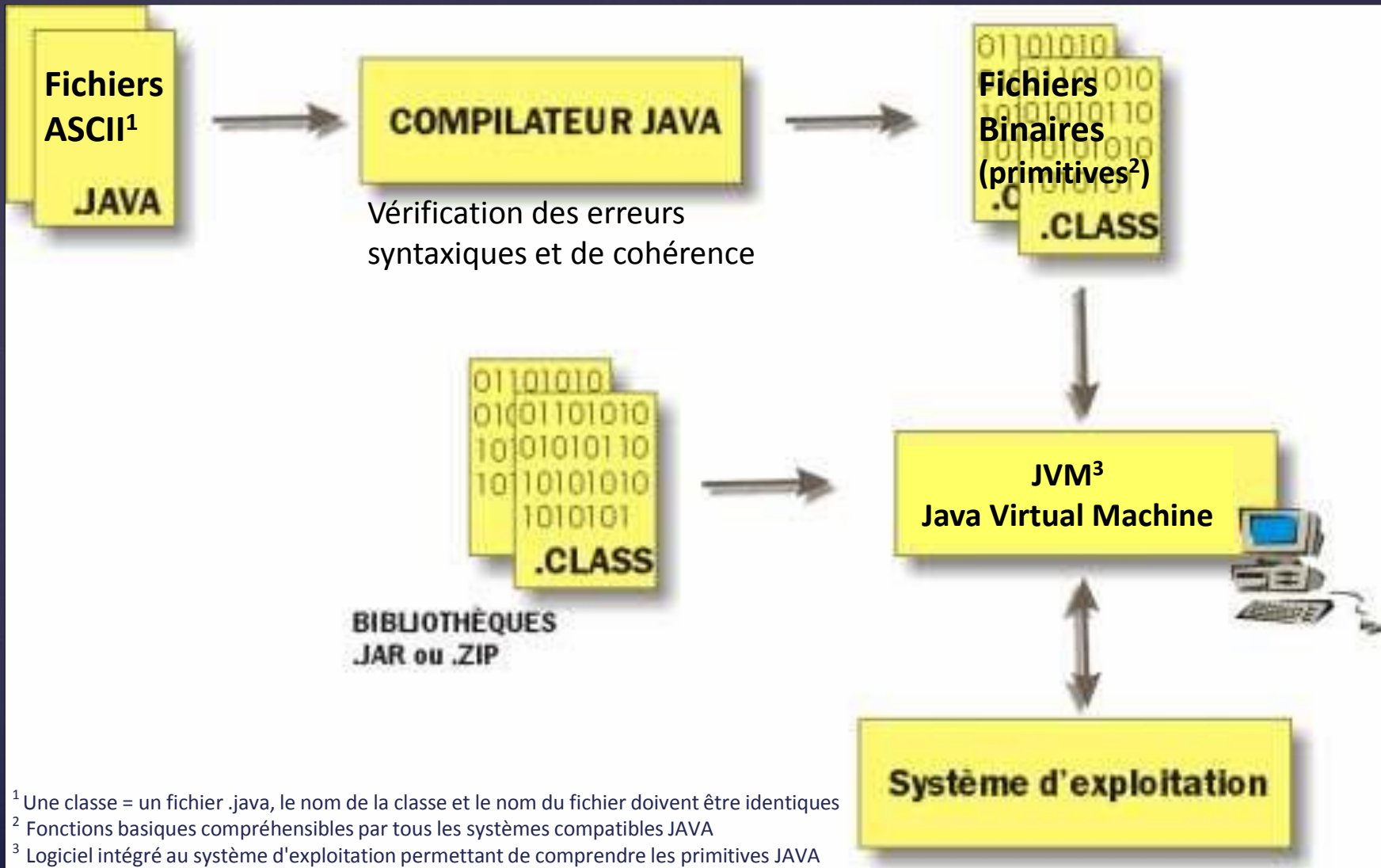
- ✓ *Les règles de nommage en JAVA*
- ✓ *L'indentation*
- ✓ *Les commentaires et documentation*

Premier programme

- ✓ *Compilation du programme*
- ✓ *Exécution du programme*

L'API Java et la documentation

L'environnement d'exécution d'un programme



Compilateur vérifie

- ✓ *Les erreurs de syntaxe,*
- ✓ *L'incohérence entre classes et objets*

Sécurisation

- ✓ *Exécution impossible tant que toutes les erreurs du langage ne sont pas levées.*

Comparaison

- ✓ *En C/C++ , pas de vérification de la mémoire et des pointeurs.*
- ✓ *En java, on utilise que des références aux objets.*

Anecdote : Objet, classe, pointeur et référence

Objet & classe

- ✓ *Objet* : Ma pizza préférée « Bella Italia »
- ✓ *Classe* : Recette de ma pizza

L'objet pizza contient beaucoup d'ingrédient et est donc volumineuse et difficile à recopier.



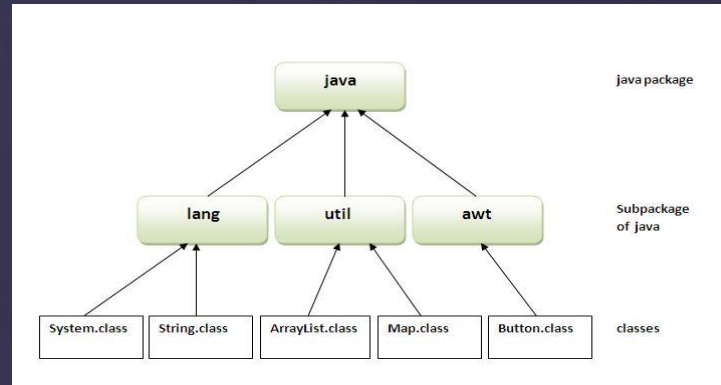
Pointeur & référence

- ✓ *Pointeur* : Adresse de la pizzeria qui fait ma pizza préférée, facile à recopier.
- ✓ *Référence* : Nom de la pizzeria « Chez Tony » facile à utiliser.



L'utilisation des packages

Organisation en packages indépendants des répertoires physiques



Librairie java regroupe les classes d'un même package dans des archives .jar ou .zip

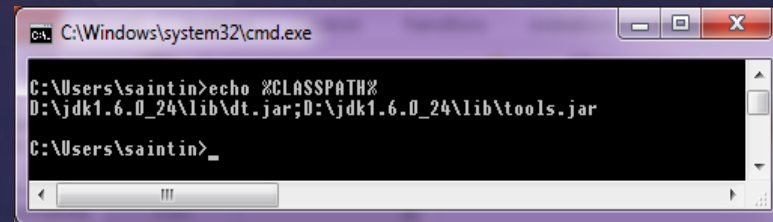
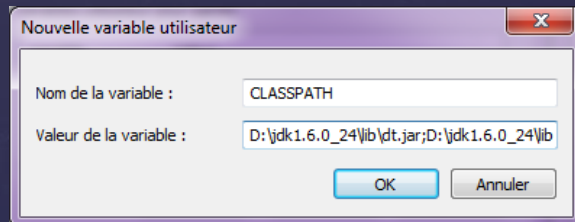
J2SE (Java 2 Standard Edition) contient nativement plus de 5000 classes utilisables dans vos programmes.

```
1  /* Extrait d'un programme JAVA */
2  package fr.soleil.passerelle.test;
3
4  import java.io.*;           // Importe toutes les classes de java.io
5
6  import java.util.GregorianCalendar; // Importe seulement la classe GregorianCalendar
7
8
9  public class MyClasse {
10
11      private File m_File;
12      private GregorianCalendar m_Calendar;
13
14  }
```

- ✓ *Liste les répertoires physiques contenant les classes.*
- ✓ *Il permet à la JVM, le chargement de toutes ces classes au lancement du programme par le Class Loader.*

Définition selon 2 méthodes :

- ✓ *La variable d'environnement du même nom*



- ✓ *Le paramètre `-classpath` ou `-cp` de la JVM*

```
java -cp %CLASSPATH% fr.soleil.gui.MyPanel
```

Les règles de nommage en Java

Règles générales

- ✓ *Ne pas mettre de caractères spéciaux, ni d'espace*
- ✓ *Ne suivez pas les conseils donnés ici :*
- ✓ *https://fr.wikipedia.org/wiki/Code_impénétrable*

Gné ?



Règles de nommage de package

- ✓ *Commence par le code pays (fr, com, be, uk...)*
- ✓ *Puis l'institution ou l'entreprise (soleil, ifa...)*
- ✓ *Le nom du projet*
- ✓ *Puis le module (view, model, lib, gui ...)*

```
Set _=Set%p%
%_ % _=If Exist%p%
%_ % _=Echo%p%

...//...

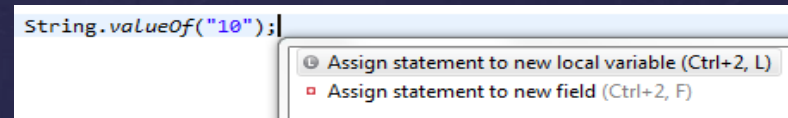
%% pAkhenatonp>>%2
%_ % i%2 %_ % F%p%
Ren %2 i%2%p%
%% @%% Off>%2%p%
Find "p"<%_ %>%2
%% %_ % i%2 %_ % E>>%2%p%
```

Règles de nommages des classes

- ✓ *Commence par une Majuscule pour chaque mot*
- Ex : MaClasse*

Règles de nommages des variables

- ✓ *Eviter les noms à rallonge ex: maSuperVariableDeLaMortQuiTue*
- ✓ *Donner des noms significatifs (pas toto)*
- ✓ *Commence par une minuscule (Eclipse Ctrl + Shift + 1)*



Règles générales

- ✓ *Ne pas mettre de caractères spéciaux, ni d'espace*

Règles de nommage de package

- ✓ *Commence par le code pays (fr, com, be, uk...)*
- ✓ *Puis l'institution ou l'entreprise (soleil, ifa...)*
- ✓ *Le nom du projet*
- ✓ *Puis le module (view, model, lib, gui ...)*

Règles de nommages des classes

- ✓ *Commence par une Majuscule pour chaque mot*
Ex : MaClasse

Règles de nommages des variables

- ✓ *Donner des noms significatifs (pas toto)*
- ✓ *Commence par une minuscule (Eclipse Ctrl + Shift + F)*

Importance pour le code

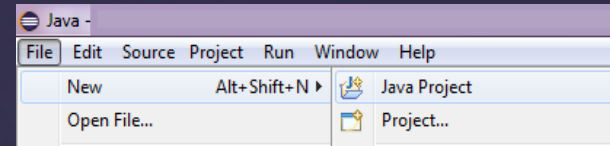
- ✓ *Documentation (javadoc voir plus loin)*
- ✓ *Lisibilité*
- ✓ *Maintenance*

Astuces sous Eclipse

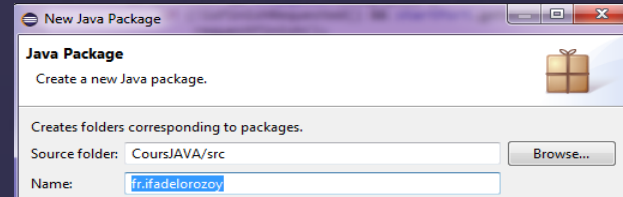
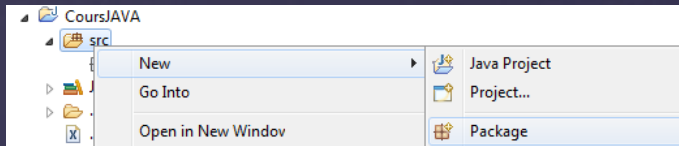
- Saisir /* puis entrée (complétion)
- Saisir /** puis entrée au dessus d'une méthode (complétion javadoc)
- CTRL + SHIFT + F (indentation)

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     /**
6      * Commentaire pour javadoc
7      * Point d'entrée du programme
8      * @param args, argument du programme
9      */
10
11     public static void main(String[] args) {
12
13         // Commentaire sur une seule ligne
14         // Affiche le message "Mon premier programme"
15
16         System.out.println("Mon premier programme");
17
18         /*Commentaire
19          sur plusieurs
20          lignes */
21         /* Sortie
22          * De
23          * JVM
24          */
25         System.exit(0);
26     }
27
28 }
```

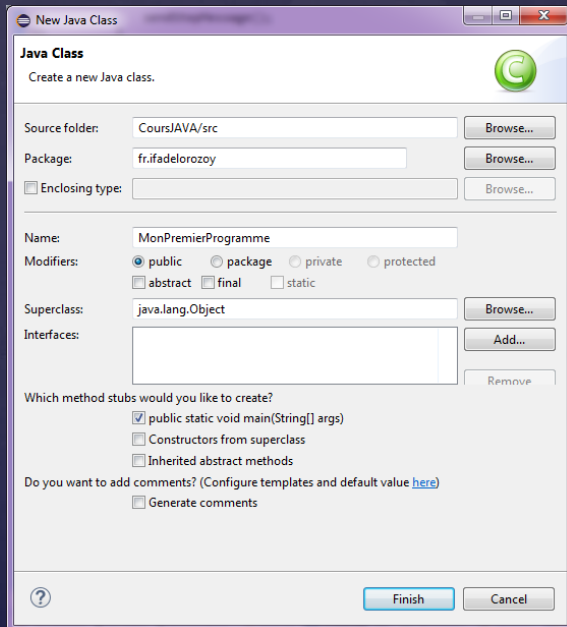

1 – File > New > Java Project



2 – Click droit > New > Package



3 – Click droit > New > Class



- Norme Java nommage des classes
- Java est case sensitive
- Raccourci Ctrl + Espace (complétion)
- Raccourci « syso »
- Méthode main statique¹

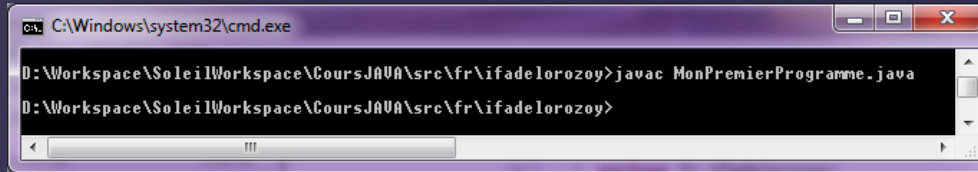
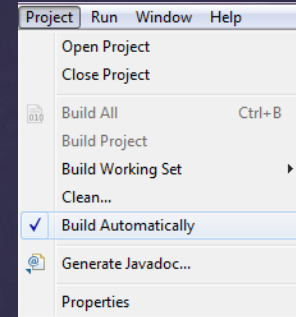
```
1 package fr.ifadelorozoy;  
2  
3 public class MonPremierProgramme {  
4  
5     public static void main(String[] args) {  
6         System.out.println("Mon premier programme");  
7         System.exit(0);  
8     }  
9  
10 }
```

¹ méthode appartenant à la classe et non à un objet (notion vue plus tard).

La compilation et exécution du programme

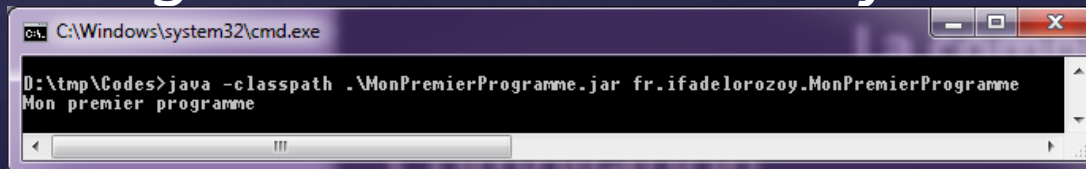
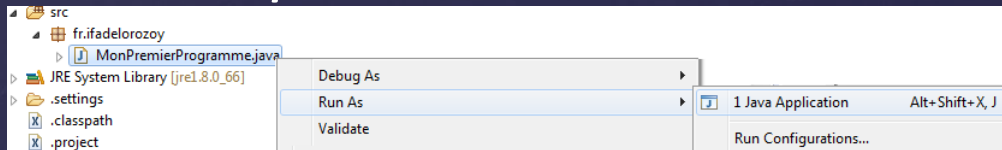
Compilation

- ✓ *Sous Eclipse > Project > Build ...*
- ✓ *En ligne de commande > javac*



Exécution

- ✓ *Sous Eclipse > Click droit > Run As > Java Application*
- ✓ *En ligne de commande > java*



- Fourni avec l'environnement de base SDK.
- API signifie « Application Programming Interface »
- Elle contient une vraie mine d'or pour les développeurs

Utilisation de la javadoc (<https://docs.oracle.com/javase/8/docs/api/>)

The screenshot shows the Java API documentation for the `String` class. The left sidebar lists various Java packages, with `String` selected under `java.lang`. The main content area displays the class details for `String`.

Annotations on the screenshot point to the following elements:

- Nom de la classe**: Points to the `String` class name.
- Chaîne d'héritage**: Points to the inheritance chain: `java.lang.Object` and `java.lang.String`.
- Signature de la classe**: Points to the class signature: `public final class String`.

The `String` class is shown as `compact1, compact2, compact3` and `java.lang`. It implements `Serializable`, `CharSequence`, and `Comparable<String>`.

The class signature is: `public final class String` extends `Object` implements `Serializable`, `Comparable<String>`, `CharSequence`.

The methods section is divided into tabs: **All Methods**, **Static Methods**, **Instance Methods**, **Concrete Methods**, and **Deprecated Methods**. The **All Methods** tab is selected.

Modifier and Type	Method and Description
char	<code>charAt(int index)</code> Returns the char value at the specified index.
int	<code>codePointAt(int index)</code> Returns the character (Unicode code point) at the specified index.
int	<code>codePointBefore(int index)</code> Returns the character (Unicode code point) before the specified index.
int	<code>codePointCount(int beginIndex, int endIndex)</code> Returns the number of Unicode code points in the specified text range of this String.
int	<code>compareTo(String anotherString)</code> Compares two strings lexicographically.
int	<code>compareToIgnoreCase(String str)</code> Compares two strings lexicographically, ignoring case differences.
String	<code>concat(String str)</code> Concatenates the specified string to the end of this string.
boolean	<code>contains(CharSequence s)</code> Returns true if and only if this string contains the specified sequence of char values.

Introduction

Les entiers

Les nombres à virgule flottante

Les caractères de type char

Les booléens

Les chaînes

- ✓ *Concaténation*
- ✓ *Sous-chaînes*
- ✓ *Manipulation de chaînes*
- ✓ *Référence à un objet chaîne*

Les énumérations

Principe

- ✓ *Toute variable doit être déclarée pour réserver un espace mémoire.*
- ✓ *Cet espace dépend du type.*
- ✓ *Une variable se définit par un **type**, un **nom** et un **modificateur**.*
- ✓ *Il existe 8 types primitifs* (Types prédéfinis par l'environnement JAVA (toutes versions confondues) :
 - *6 numériques (4 types d'entiers et 2 types de réels à virgule flottante)*
 - *1 pour les caractères (char)*
 - *1 booléen (boolean)*

Voici un tableau représentant les 4 types d'entiers :

Type	Taille en mémoire	Intervalles (limites incluses)
byte	1 octet (8 bits)	-128 à 127 (-2^7 à 2^{7-1})
short	2 octets (16 bits)	-32768 à 32767 (-2^{15} à 2^{15-1})
int	4 octets (32 bits)	-2147483648 à 2147483647 (-2^{31} à 2^{31-1})
long	8 octets (64 bits)	-9223372036854775808 à 9223372036854775807 (-2^{63} à 2^{63-1})

Pour déclarer ce type d'entiers, voici un panel d'exemples :

```
7 private int monEntier1 = 123478; // Correct
8 private int monEntier2 = 12.0 ; // Incorrect , erreur de compilation
9
10 private short monEntier3 = 32000; // Correct
11 private short monEntier4 = 33000; // Incorrect , erreur de compilation
12
13 //Les entiers longs ont un suffixe L
14 private long monEntier5 = 3000000000L ; // Correct
15 private long monEntier6 = 3000000000 ; // Incorrect , erreur de compilation
16
17 private byte monEntier7 = -40; // Correct
18 private byte monEntier8 = 128; // Incorrect , erreur de compilation
19
20 //Les entiers hexadécimaux sont codés ainsi
21 int monEntierHexa = 0x12A ;
```

Les 2 types de nombres réels :

Type	Taille en mémoire	Intervalles (limites incluses)
float	4 octets (32 bits)	-3.40282347E ³⁸ à 3.40282347E ³⁸
double	8 octets (64 bits)	-1.79769313486231570E ³⁰⁸ à 1.79769313486231570E ³⁰⁸

Voici 2 exemples de déclaration :

```
/*si vous ne spécifiez pas le type flottant à la déclaration,  
* c'est un double qui sera crée.  
*/  
float monNombre = 12.34F ;      // F : précise qu'il s'agit d'un flottant  
  
double monNombre2 = 13.45E14 ;
```

- ✓ Utilisation des float : si la vitesse de calcul et l'encombrement de la mémoire sont prioritaires par rapport à la précision.
- ✓ **Attention** à ne pas confondre les types primitifs et les classes numériques équivalentes du package java.lang : int et Integer, float et Float, double et Double ...

Les caractères de type char

Le type char désigne des caractères en représentation Unicode6. Un caractère Unicode¹ est constitué du caractère d'échappement \u et d'un chiffre en hexadécimal sur 2 octets :

```
char monCaractere = '\u2122' ; // Caractère de marque déposée TM
```

Les caractères spéciaux sont classés dans le tableau suivant :

<i>Caractère d'échappement</i>	<i>Signification</i>	<i>Valeur Unicode</i>
<i>\b</i>	<i>Effacement arrière</i>	<i>\u0008</i>
<i>\t</i>	<i>Tabulation horizontale</i>	<i>\u0009</i>
<i>\n</i>	<i>Saut de ligne</i>	<i>\u000a</i>
<i>\r</i>	<i>Retour chariot</i>	<i>\u000d</i>
<i>\"</i>	<i>Double apostrophe</i>	<i>\u0022</i>
<i>\'</i>	<i>Apostrophe</i>	<i>\u0027</i>
<i>\\</i>	<i>Antislash</i>	<i>\u005c</i>

¹ Pour tout savoir sur l'Unicode , <http://www.unicode.org> .

Jeu de caractères offrant plus de possibilités que l'ASCII puisqu'il est possible de distinguer 65536 caractères différents contre 256 pour l'ASCII. Unicode est prévu pour coder l'ensemble des lettres de tous les pays

Le type primitif **boolean** peut avoir 2 valeurs : *true* ou *false*.

Utilisé dans les instructions type *if...else ...*

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     private static boolean boolean1 = false;
6
7     public static void main(String[] args) {
8         System.out.println("Mon premier programme");
9
10        if(boolean1){
11            System.out.println("Mon boolean est vrai");
12        }
13    }
14 }
```

Attention à ne pas confondre le type primitif boolean et la classe équivalente du package java.lang : Boolean

Contrairement aux types précédents, il n'existe pas en JAVA, de type primitif gérant des chaînes de caractères. Par contre, JAVA propose un objet¹ *String*. En conséquence, toute chaîne est une **instance** de la classe *String*.

Exemple de concaténation de chaîne :

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6         String maChaine = "Bonjour ";
7         maChaine = maChaine + "à tous !";
8
9         System.out.println(maChaine);
10
11         System.exit(0);
12     }
13 }
14
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (17 sept. 2016 00:20:27)
Bonjour à tous !

¹ Pour Les notions objets les *spécificités* et les *différentes manières d'utiliser des objets* seront vus ultérieur

TP n°1 : Manipulation des chaînes

Grâce à la javadoc (F3 sur la classe String) et à la complétion d'Eclipse, réaliser les manipulations de chaînes suivantes dans le main de votre classe.

MonPremierProgramme et commenter votre code :

1. Déclarer une chaîne maChaîne initialisée à « Bonjour »
2. Afficher la longueur de cette chaîne.
3. Afficher la sous-chaîne « Bon » et « jour »
4. Afficher la chaîne en lettre CAPITALE
5. Afficher le caractère situé à l'indice 3
6. Afficher l'indice du premier o
7. Afficher Bonsoir en remplaçant « jour » par « soir »

TP n°1 : Manipulation des chaînes solution

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6
7         // 1 declaration et initialisation de ma chaîne
8         String maChaine = "Bonjour";
9
10        // 2 Afficher la longueur de cette chaîne
11        System.out.println("Longueur de ma chaîne = " + maChaine.length());
12
13        // 2 Afficher la sous-chaîne « Bon »
14        System.out.println("Sous-chaîne 1 = " + maChaine.substring(0, 3));
15
16        // 2 Afficher la sous-chaîne « jour »
17        System.out.println("Sous-chaîne 2 = " + maChaine.substring(3, 7));
18        // Autre solution plus simple
19        System.out.println("Sous-chaîne 2bis = " + maChaine.substring(3));
20
21        // 4 - Afficher la chaîne en lettre CAPITALE
22        System.out.println("ma chaîne en CAPITALE= " + maChaine.toUpperCase());
23
24        // 5 - Afficher le caractère situé à l'indice 3
25        System.out.println("Caractère à l'indice 3 = " + maChaine.charAt(3));
26
27        // 6 - Afficher l'indice du premier o
28        System.out.println("Indice du premier o = " + maChaine.indexOf("o"));
29
30        // 7 - Afficher Bonsoir en remplaçant « jour » par « soir »
31        System.out.println("Remplacer jour par soir = " + maChaine.replaceFirst("jour", "soir"));
32
33        System.exit(0);
34    }
35 }
```

Problems @ Javadoc Declaration Search Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files

```
Longueur de ma chaîne = 7
Sous-chaîne 1 = Bon
Sous-chaîne 2 = jour
Sous-chaîne 2bis = jour
ma chaîne en CAPITALE= BONJOUR
Caractère à l'indice 3 = j
Indice du premier o = 1
Remplacer jour par soir =Bonsoir
```

Les énumérations sont un ajout de Java 5. Une énumération est un type de données particulier, dans lequel une variable ne peut prendre qu'un nombre restreint de valeurs.

Ces valeurs sont des constantes nommées, par exemple MADAME, MADEMOISELLE, MONSIEUR.

Exemple de déclaration:

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     enum Civilite {MADAME, MADEMOISELLE, MONSIEUR} ;
6
7     public static void main(String[] args) {
8
9         boolean female = true;
10        System.out.println(female ? Civilite.MADAME : Civilite.MONSIEUR);
11        System.out.println(!female ? Civilite.MADAME : Civilite.MONSIEUR);
12        System.exit(0);
13    }
14 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (

MADAME

MONSIEUR

Conversions de type

- ✓ *Conversion implicite*
- ✓ *Conversion explicite*
- ✓ *Cas particulier de résultat tronqué*

Constantes

Opérateurs

- ✓ *Opérateurs arithmétiques,*
- ✓ *Opérateurs d'incrémentement*
- ✓ *Opérateurs relationnels*
- ✓ *Opérateurs binaires*
- ✓ *Opérateur ternaire (affectation conditionnelle)*
- ✓ *Hiérarchie des opérateurs*

Conversion implicite

```
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6
7         int a = 4;
8         double b = 3.5;
9
10        b = b * a; //converti implicitement a en double
11        System.out.println("Resultat=" + b);
12        a = b * a; //erreur de compilation car perte de précision
13
14        System.exit(0);
15    }
16 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (17 s)
Resultat=14.0

Conversion explicite

```
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6
7         int a = 3;
8         float b = 4.3F;
9
10        a = (int) b * a; //converti explicitement b en int
11        /*
12         * b est converti en entier avant l'opération
13         * Il y a cependant une perte d'information
14         */
15
16        System.out.println("Resultat=" + a);
17    }
18 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (17 s)
Resultat=12

Une conversion explicite est aussi appelée transtypage et permet d'éviter les erreurs de compilation.

Cas particulier de résultat tronqué

```
7     short a;
8     int b= 53456;
9     a = (short) b; // Conversion de b en short, le resultat sera tronqué
10    System.out.println("Resultat=" + a);
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (17 s)
Resultat=-12080

Pour déclarer une constante, il suffit d'employer le modificateur final devant la déclaration de la variable.

Par convention, les constantes sont en majuscules.

```
7
8 public static void main(String[] args) {
9
10     final double EURO = 6.55957;    // Taux de change pour le franc
11
12     System.out.println("Euro=" + EURO);
13     System.out.println("Pi=" + Math.PI);
14     System.out.println("E=" + Math.E ); ;
15 }
```

Console

```
<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe
Euro=6.55957
Pi=3.141592653589793
E=2.718281828459045
```

Il en existe 5 :

- +: addition
- - : soustraction
- *: multiplication
- / : division
- %: reste de la division

```
int a = 23 + 12 ;    // a contient 35
int b = a * 2;       // b contient 70
int c = b / 3;        // c contient 23
double d = b / 3;     // d contient 23.0
double e = b / 3.0;   // e contient 23.333333333333332
int f = 50 % 12;      // f contient le reste de la division = 2
```

Les operateurs d'incrémentation

Opérateurs arithmétiques prédéfinis pour **incrémenter** (ajouter 1) ou **décrémenter** (retirer 1) des variables. Selon l'endroit où se situe l'opérateur (avant ou après le nom de la variable), l'opération est effectuée avant ou après l'évaluation de l'expression.

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5
6
7
8 public static void main(String[] args) {
9
10     int x = 0;
11
12     x++; // Incrémentation de x
13     System.out.println("Valeur de x = " + x );
14     x--; // Décrémentation de x
15     System.out.println("Valeur de x = " + x );
16
17     System.exit(0);
18 }
19 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18 s)
Valeur de x = 1
Valeur de x = 0

```
7
8 public static void main(String[] args) {
9
10     int x = 10;
11     int y = x++; //Affecte la valeur de x à z avant l'incrémentation de x
12     int z = ++x; //Affecte la valeur de x à y après l'incrémentation de x
13
14     System.out.println("Valeur de x = " + x );
15     System.out.println("Valeur de y = " + y );
16     System.out.println("Valeur de z = " + z );
17
18     System.exit(0);
19 }
20 }
21 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18 s)
Valeur de x = 12
Valeur de y = 10
Valeur de z = 12

Attention à ne pas trop en abuser :

- ✓ Illisibilité du code
- ✓ Source d'erreur possible.

Les operateurs relationnels

Il en existe 8 : <, <=, >, >=, ==, !=, || (OR) et && (AND)

Ils permettent de comparer des variables entre elles. Le résultat de la comparaison est un booléen dont les deux valeurs possibles sont *true* (Vrai) ou *false* (Faux) .

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6
7         System.out.println("3 < 4 = " + (3 < 4));
8         System.out.println("5 <= 4 = " + (5 <= 4));
9         System.out.println("5 != 4 = " + (5 != 4));
10        System.out.println("false || true = " + (false || true));
11        System.out.println("true && false = " + (true && false));
12        System.out.println("Double(3.0) == Double(3.0) " + (new Double(3.0) == new Double(3.0)) );
13        System.out.println("Double(3.0) equals Double(3.0) " + ((new Double(3.0)).equals(new Double(3.0)))) );
14
15        System.exit(0);
16    }
17 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18 sept. 2016 23:11:20)

```
3 < 4 = true
5 <= 4 = false
5 != 4 = true
false || true = true
true && false = false
Double(3.0) == Double(3.0) false
Double(3.0) equals Double(3.0) true
```

Attention : - avec les Objets, la comparaison se fait sur la référence
- ne pas confondre = pour l'affectation et == pour le test d'égalité

Il en existe 4 : | (OR) , &(AND), >> et <<

Utiles pour manipuler des données au niveau machine (registres internes d'un processeur, mémoire ...). Il existe des techniques de masquage de bits ces opérateurs.

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6
7         int a=9;    // 9 en décimal vaut 1001 en binaire
8         System.out.println("a & 4 = " + (a & 4)); //1001 & 0100 = 0000 = 0
9         System.out.println("a & 8 = " + (a & 8)); //1001 & 1000 = 1000 = 8
10        System.out.println("a | 4 = " + (a | 4)); //1001 | 0100 = 1101 = 13
11        System.out.println("a | 8 = " + (a | 8)); //1001 | 1000 = 1001 = 9
12        System.out.println("a ^ 8 = " + (a ^ 8)); //1001 ^ 1000 = 0001 = 1
13        System.out.println("a >> 1 = " + (a >> 1)); //1001 => 0100 = 4 on décale à droite de 1 bit
14        System.out.println("a << 2 = " + (a << 2)); //0000 1001 => 0010 0100 = 36 on décale à gauche de 2 bits
15        System.exit(0);
16    }
17 }
18
```

Console

```
<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18 sept. 2016 23:34:23)
a & 4 = 0
a & 8 = 8
a | 4 = 13
a | 8 = 9
a ^ 8 = 1
a >> 1 = 4
a << 2 = 36
```

Opérateur ternaire (affectation conditionnelle)

Lorsque les instructions du type if ou else consistent à donner une valeur à une variable, il est possible de faire un test avec une structure beaucoup moins lourde grâce à l'opérateur ternaire :

variable = condition ? valeur_si_vrai : valeur_si_faux

Par exemple :

```
3 public class MonPremierProgramme {
4
5  public static void main(String[] args) {
6      // Instruction standard if else pour affectation
7      int min;
8      int a = 6;
9      int b = 8;
10
11      if (a < b) {
12          min = a;
13      }
14      else {
15          min = b;
16      }
17
18      System.out.println("min = " + min);
19      System.exit(0);
20  }
21 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre...
min = 6

=>

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5  public static void main(String[] args) {
6      // Instruction avec operateur ternaire
7      int min;
8      int a = 6;
9      int b = 8;
10
11      min = a < b ? a : b;
12
13      System.out.println("min = " + min);
14      System.exit(0);
15  }
16 }
17 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program F...
min = 6

Comme dans tous les langages de programmation, il est préférable d'employer des parenthèses pour indiquer l'ordre dans lequel les opérations doivent être accomplies.

Voici un tableau récapitulant cette hiérarchie :

1. `[ ] , ( )`
2. `! , ++ , --`
3. `* , / , %`
4. `+ , -`
5. `<< , >> , >>>`
6. `< , <= , > , >=`
7. `== , !=`
8. `&`
9. `^` (ou exclusif)
10. `|`
11. `&&`
12. `||`
13. `?:`

Instruction if ... else

Boucle while

Boucle for

Instruction switch ... case

Return

Elle permet d'effectuer une instruction ou un bloc d'instructions selon des conditions précises. Si la condition est vraie, le bloc d'instructions est exécuté sinon, le programme passe à la suite. Voici une illustration de la forme :

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6         boolean condition = true;
7
8         if (condition)
9         {
10             //bloc d'instructions
11             System.out.println("La condition est réalisée");
12         }
13         else {
14             System.out.println("La condition n'est pas réalisée");
15         }
16     }
17 }
18
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\j
La condition est réalisée

Lorsque le bloc d'instruction ne contient qu'une seule instruction, vous pouvez omettre les accolades.

Elle permet d'exécuter des blocs d'instructions de manière répétitives et cela, tant qu'une condition est vraie. Voici la forme de cette boucle :

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6         boolean condition = true;
7         int i = 0;
8         while (condition)
9         {
10             i++;
11             System.out.println("itération n°" + i);
12             if(i == 10){
13                 condition = false;
14             }
15         }
16         System.out.println("Fin de boucle");
17         System.exit(0);
18     }
19 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\J
itération n°1
itération n°2
itération n°3
itération n°4
itération n°5
itération n°6
itération n°7
itération n°8
itération n°9
itération n°10
Fin de boucle

Elle permet d'effectuer une boucle avec un nombre d'itérations (passages dans le bloc) défini à l'avance :

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6         for (int i = 0 ; i <= 10 ; i++)
7         {
8             System.out.println("itération n°" + i);
9         }
10        System.out.println("Fin de boucle");
11        System.exit(0);
12    }
13 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe

itération n°0
itération n°1
itération n°2
itération n°3
itération n°4
itération n°5
itération n°6
itération n°7
itération n°8
itération n°9
itération n°10
Fin de boucle

```
1 package fr.ifadelorozoy;
2
3 public class MonPremierProgramme {
4
5     public static void main(String[] args) {
6         String[] array = new String[]{"chaine1", "chaine2", "chaine3"};
7         for (String chaine : array)
8         {
9             System.out.println(chaine);
10        }
11        System.out.println("Fin de boucle");
12        System.exit(0);
13    }
14 }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe

chaine1
chaine2
chaine3
Fin de boucle

Instruction switch ... case

Lorsque le nombre d'états différents à tester d'une variable est trop important, il est préférable d'utiliser la structure ***switch..case..break*** que d'enchaîner des instructions if.

```
3 public class MonPremierProgramme {
4
5     public enum Civilite {
6         MONSIEUR, MADAME, MADemoiselle
7     };
8
9     public static void main(String[] args) {
10
11         Civilite civilite = Civilite.MADAME;
12
13         switch (civilite) {
14
15             case MONSIEUR:
16                 System.out.println("C'est un homme");
17                 break;
18
19             case MADemoiselle:
20                 System.out.println("C'est une femme");
21                 break;
22
23             case MADAME:
24                 System.out.println("C'est une femme mariée");
25                 break;
26
27             default:
28                 System.out.println("C'est un extra terrestre");
29
30         }
```

Console

<terminated> MonPremierProgramme [Java Application] C:\Program Files\Java\jre1.8.0_6
C'est une femme mariée

Les types compatibles avec cette instruction sont les :

- *char, int, byte, short,*
- *énumération depuis java 5*
- *String depuis java 7*

Attention, si le *break* est omis toutes les instructions qui suivent sont exécutées.

L'instruction **return**, permet de :

- terminer l'exécution d'une méthode
- retourner une valeur

```

2
3 public class ReturnInstruction {
4
5     private static void maMethodeReturn(boolean affichageConsole){
6         if(affichageConsole){
7             System.out.println("Je suis dans maMethodeReturn !");
8             return;
9         }
10        //Code non atteint si affichageConsole = true à cause du return
11        System.out.println("affichage console désactivé");
12    }
13
14    private static String maMethodeReturnValue(){
15        //retourne une valeur texte
16        return "Valeur de retour";
17    }
18
19    public static void main(String[] args) {
20        //Appel maMethodeReturn avec affichage console
21        maMethodeReturn(true);
22        //Appel maMethodeReturn sans affichage console
23        maMethodeReturn(false);
24
25        //Affiche le resultat de maMethodeReturnValue
26        System.out.println("maMethodeReturnValue=" + maMethodeReturnValue());
27
28        //Return rendant le code qui suit inatteignable
29        return;
30        //Affiche le resultat de maMethodeReturnValue
31        Unreachable code:out.println("Code inatteignable =");//Code mort
32    }
33 }
34

```

Attention un return rendant un code inatteignable provoque une erreur de compilation.

Problems Javadoc Declaration Search Console Progress History Synchroniz

<terminated> ReturnInstruction [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (22 sept.)

Je suis dans maMethodeReturn !

affichage console désactivé

maMethodeReturnValue=Valeur de retour

Objectif savoir utiliser :

- La boucle **for**
- Les **opérateurs arithmétiques**
- Les méthodes de la classe **String** grâce à la complétion

Créer une classe NombreMagique :

1. Déclarer une constante MAGIC_NUMBER=142857
2. Multiplier MAGIC_NUMBER par 1, 2 , 3 , 4 et 5
3. Convertir chaque résultat en chaîne de caractère.
4. Rechercher l'indice du caractère « 1 » (_ _ 1 _ _)
5. Créer une sous-chaîne de « 1 » jusqu'à la fin (_ _ 1 _ _)
6. Créer une sous-chaîne du début jusqu'à « 1 » (_ _ 1 _ _)
7. Concaténer les 2 sous-chaîne et afficher le résultat
8. Comparer MAGIC_NUMBER avec le résultat

TP n°2 Le chiffre magique solution

```
1 package fr.ifadelorozoy;
2
3 public class NombreMagique {
4
5     //Déclaration d'une constante variable de classe MAGIC_NUMER
6     private static final int MAGIC_NUMBER = 142857;
7
8     public static void main(String[] args) {
9         // Declaration de chaine nulle hors de la boucle for
10        String maChaine = null;
11        String subString1 = null;
12        String subString2 = null;
13        String stringConcat = null;
14        for (int i=1; i<6;i++){
15            //Multiplication du chiffre magique par i
16            int resultXi = i*MAGIC_NUMBER;
17
18            // conversion du resultat en chaine de caractere
19            maChaine = String.valueOf(resultXi);
20
21            //Recuperation de l index du caractere 1
22            int indexOf1 = maChaine.indexOf("1");
23
24            //Sous chaine du caractere 1 jusqu a la fin
25            subString1 = maChaine.substring(indexOf1, maChaine.length());
26
27            //Sous chaine du debut jusqu au caractere 1
28            subString2 = maChaine.substring(0, indexOf1);
29
30            //Concatenation des 2 sous chaines
31            stringConcat = subString1 + subString2;
32
33            //Test est ce que la chaine est égale au nombre magique ?
34            boolean equals= stringConcat.equals(String.valueOf(MAGIC_NUMBER));
35
36            // Affiche le resultat de la concatenation et du test
37            System.out.println(stringConcat + " égal au MAGIC NUMBER ? " + equals);
38        }
39
40        System.exit(0);
41    }
42
43 }
```

```
<terminated> NombreMagique [Java Application] C:\Program Files\Java
142857 égal au MAGIC NUMBER ? true
142857 égal au MAGIC NUMBER ? true
142857 égal au MAGIC NUMBER ? true
142857 égal au MAGIC NUMBER ? true
142857 égal au MAGIC NUMBER ? true
```

Déclaration d'un tableau

La classe utilitaire `java.util.Array`

- ✓ *Comparaison d'un tableau de `String`*
- ✓ *TP n°3 utilisation de la classe `Array`*

Les structure de données

- ✓ *Qu'est-ce-qu'une structure de données ?*
- ✓ *Qu'est-ce-qu'une collection ?*
- ✓ *Le framework des collections*
- ✓ *Les tables de hachage*
- ✓ *Structures récursives*

Déclaration de tableaux

Ils permettent de stocker les données dans un ensemble et d'accéder à ces données par l'intermédiaire d'indices.

Un tableau se comporte exactement comme un objet. Ces derniers peuvent contenir d'autres objets.

```
4
5 public static void main(String[] args) {
6
7     // Declaration d'un tableau de 50 entiers
8     int [] monTableau = new int[50];
9
10    int monTableau2[] = new int[50];
11
12
13    // Declaration d'un tableau de 4 chaînes
14    String [] monTableauDeChaine = new String[4];
15
16    //Affichage de la taille du tableau de chaîne
17    int taille = monTableauDeChaine.length;
18    System.out.println("Taille du tableau de chaîne = " + taille);
19
20    //Affectation du premier élément du tableau de chaîne => indice = 0
21    monTableauDeChaine[0] = "Ma premiere chaîne";
22    //Affectation du second élément du tableau de chaîne => indice = 1
23    monTableauDeChaine[1] = "Ma seconde chaîne";
24    //Affectation du dernier élément du tableau de chaîne => indice = taille - 1
25    monTableauDeChaine[taille - 1] = "Ma dernière chaîne";
26
27    // Declaration d'un tableau de 3 chaînes
28    String [] monTableauDeChaine2 = { "Chaîne 1" , "Chaîne 2" , "Chaîne 3" };
29
30    // Boucle pour afficher les valeurs
31    for (String chaine : monTableauDeChaine2){
32        System.out.println(chaine);
33    }
34 }
```

Problems ▣ Javadoc ▣ Declaration 🔍 Search Console ⌵ Progress 📄 History 🔄 Synchronize

<terminated> MonPremierProgramme [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (19 sept. 2016)

Taille du tableau de chaîne = 4
Chaîne 1
Chaîne 2
Chaîne 3

Lors qu'on nécessite un tableau de taille variable, on préférera utiliser les collections.

La classe Array : Comparaison de tableaux

ATTENTION : Les tableaux, en JAVA, sont des références cachées. Ce qui signifie qu'un objet déclaré comme un tableau est en fait une référence à cet objet et non l'objet lui-même.

```
1 package fr.ifadelorozoy;
2
3 import java.util.Arrays;
4
5 public class MonPremierProgramme {
6
7     public static void main(String[] args) {
8
9         // Declaration d'un tableau de 3 chaînes
10        String [] monTableauDeChaine1 = { "Chaine 1" , "Chaine 2" , "Chaine 3" };
11
12        // Declaration d'un tableau de 3 chaînes identiques
13        String [] monTableauDeChaine2 = { "Chaine 1" , "Chaine 2" , "Chaine 3" };
14
15        //Comparaison des références
16        boolean referenceEquals = monTableauDeChaine1.equals(monTableauDeChaine2);
17        System.out.println("Les 2 tableaux sont-il identiques en référence =" + referenceEquals);
18
19        //Comparaison du contenu
20        boolean containsEquals = Arrays.equals(monTableauDeChaine1, monTableauDeChaine2);
21        System.out.println("Les 2 tableaux sont-il identiques en contenu =" + containsEquals);
22
23        System.exit(0);
24    }
25 }
```

Problems Javadoc Declaration Search Console Progress History Synchronize Outline

<terminated> MonPremierProgramme [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (19 sept. 2016 16:59:19)
Les 2 tableaux sont-il identiques en référence =false
Les 2 tableaux sont-il identiques en contenu =true

Grâce à la javadoc (F3 sur la classe `java.util.Arrays`) et à la complétion d'Eclipse, réaliser les manipulations de tableaux de chaînes suivantes :

Créer une classe `UtilisationArray` :

1. Déclarer un tableau de 6 chaînes
2. Remplir le tableau de la chaîne « chaîne »
3. Afficher le contenu du tableau
4. Remplir les 2 dernières valeurs par « autre chaîne »
5. Afficher le contenu du tableau
6. Trier le tableau dans l'ordre ascendant (alphabétique)
7. Afficher le contenu du tableau
8. Copie des 2 premières valeurs dans un nouveau tableau
9. Afficher le contenu du nouveau tableau

TP n°3 : Manipulation des chaînes solution

```
1 package fr.ifadelorozoy;
2
3 import java.util.Arrays;
4
5 public class UtilisationArray {
6
7     public static void main(String[] args) {
8
9         // Declaration d'un tableau de 10 chaînes
10        String [] monTableauDeChaine = new String[6];
11
12        //Remplissage du tableau de "chaîne"
13        Arrays.fill(monTableauDeChaine, "chaîne");
14
15        //Affichage du tableau
16        System.out.println("monTableauDeChaine rempli par chaîne= " + Arrays.toString(monTableauDeChaine));
17
18
19        //Remplissage des 2 dernières valeurs par "autre chaîne"
20        Arrays.fill(monTableauDeChaine, monTableauDeChaine.length - 2 , monTableauDeChaine.length , "autre chaîne");
21        System.out.println("monTableauDeChaine rempli 2 dernières valeurs = " + Arrays.toString(monTableauDeChaine));
22
23        //Trie du tableau ascendant
24        Arrays.sort(monTableauDeChaine);
25        System.out.println("monTableauDeChaine trié= " + Arrays.toString(monTableauDeChaine));
26
27        //Copie des 2 premières valeurs vers un nouveau tableau
28        String[] nouveauTableau = Arrays.copyOf(monTableauDeChaine, 2);
29        System.out.println("nouveauTableau= " + Arrays.toString(nouveauTableau));
30    }
```

Problems ■ Javadoc ■ Declaration 🔍 Search 📄 Console ⌘ Progress 📅 History 🔄 Synchronize 🗺 Outline



<terminated> UtilisationArray [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (19 sept. 2016 17:49:43)

```
monTableauDeChaine rempli par chaîne= [chaîne, chaîne, chaîne, chaîne, chaîne, chaîne]
monTableauDeChaine rempli 2 dernières valeurs = [chaîne, chaîne, chaîne, chaîne, autre chaîne, autre chaîne]
monTableauDeChaine trié= [autre chaîne, autre chaîne, chaîne, chaîne, chaîne, chaîne]
nouveauTableau= [autre chaîne, autre chaîne]
```

- Object qui sert au **stockage** d'éléments auxquels on veut accéder plus tard.
- Les manipulation de ces éléments sont appelés **opérations**.

Par exemple, on peut définir les opérations suivantes :

- la lecture d'un élément.
- l'insertion d'un élément dans la structure.
- la suppression d'un élément de la structure.



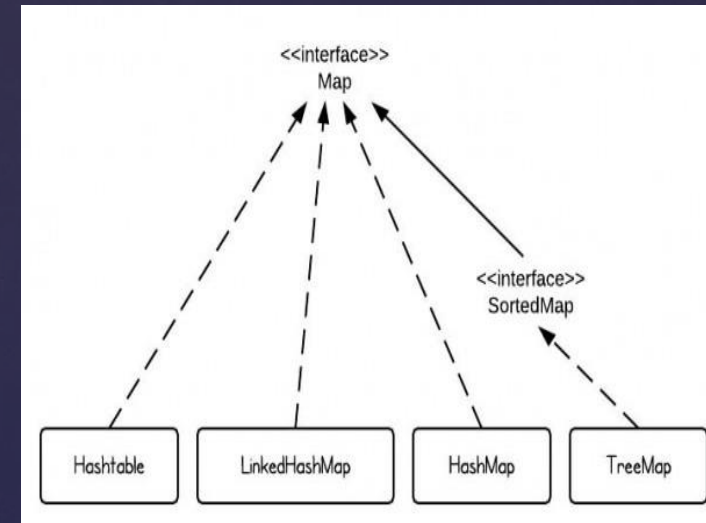
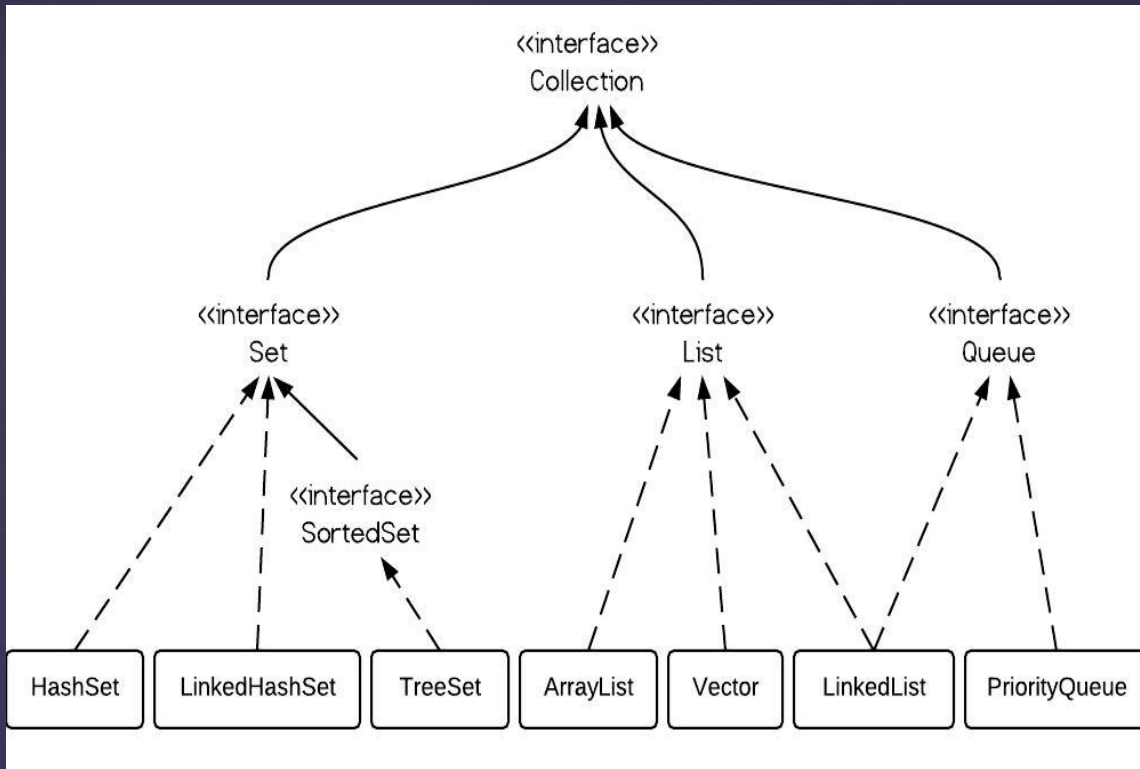
PokéDex

Les collections sont des ensembles cohérents d'objets. Elles n'ont pas de taille définitive. La collection la plus utilisée est ArrayList.

```
1 package fr.ifadelorozoy;
2
3 import java.util.ArrayList;
4 import java.util.Iterator;
5
6 public class ExempleCollection {
7     public static void main(String[] args) {
8
9         // Declaration d'une collection de type String
10        ArrayList<String> maCollection = new ArrayList<String>();
11
12        // Ajoute Lundi, mardi, mercredi
13        maCollection.add("Lundi");
14        maCollection.add("Mardi");
15        maCollection.add("Jeudi");
16
17        // Ajoute Mercredi en 3eme position
18        maCollection.add(2, "Mercredi");
19        maCollection.add("Vendredi");
20        maCollection.add("Samedi");
21        maCollection.add("Dimanche");
22
23        maCollection.add("toto");
24
25        // Retire le dernier élément de la collection
26        maCollection.remove("toto");
27
28        /* Classe Iterator est une classe d'aide permettant
29        * de parcourir la collection
30        */
31        Iterator<String> monIt = maCollection.iterator();
32
33        // Affiche à l'écran le contenu de la collection
34        String jour= null;
35        System.out.println("*****Affichage avec iterator*****");
36        while (monIt.hasNext()) {
37            jour = monIt.next();
38            System.out.println(jour);
39        }
40
41        // Affiche le contenu sans iterator
42        System.out.println("*****Affichage sans iterator*****");
43        for(String jr : maCollection){
44            System.out.println(jr);
45        }
46    }
47 }
```

```
<terminated> ExempleCollection [Java Application] C:\Program F
*****Affichage avec iterator*****
Lundi
Mardi
Mercredi
Jeudi
Vendredi
Samedi
Dimanche
*****Affichage sans iterator*****
Lundi
Mardi
Mercredi
Jeudi
Vendredi
Samedi
Dimanche
```


Le framework des collections



Set : contient des éléments qui doivent être uniques.

SortedSet : trie les éléments contenus.

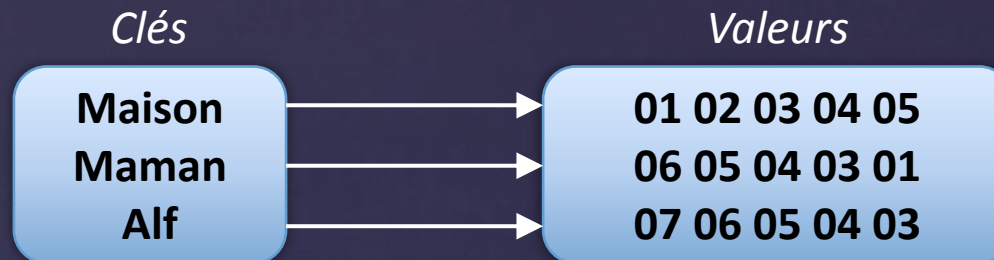
List : ordonne les éléments en autorisant les doublons.

Queue : contient des éléments insérés et retirés, FIFO (First In First Out)

Map : contient des paires : clé, valeur, ce qui facilite la recherche

Les tables de hachage

Le but du jeu est en fait de stocker des paires "clé-valeur" et de pouvoir y accéder efficacement, en utilisant seulement la clé. Un exemple de répertoire téléphonique : { nom, numéro}



```
import java.util.HashMap;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;

public class TableHashage {

    public static void main(String[] args) {

        //Declaration d'une table de hachage <Nom, Numero>
        Map<String, Long> annuaire = new HashMap<String, Long>();

        //Ajout de 3 contacts (Auto Boxing)
        annuaire.put("Maison", 0102030405L);
        annuaire.put("Maman", 0605040302L);
        annuaire.put("Alf", 0706050403L);

        //Ajout d'un faux numero
        annuaire.put("E.T", 01111111L);
        annuaire.remove("E.T");

        //Affichage du numero de Alf
        System.out.println("numéro de Alf=" + annuaire.get("Alf"));
        //Test si E.T existe
        System.out.println("numéro E.T existe =" + annuaire.containsKey("E.T"));
        //Affiche la liste des clés
        System.out.println("Affiche la liste des clés=" + annuaire.keySet());
        //Affiche tout l'annuaire
        Set<Entry<String, Long>> entrySet = annuaire.entrySet();

        for(Entry<String, Long> entry : entrySet){
            System.out.println(entry.getKey() + "=" + entry.getValue());
        }
    }
}
```

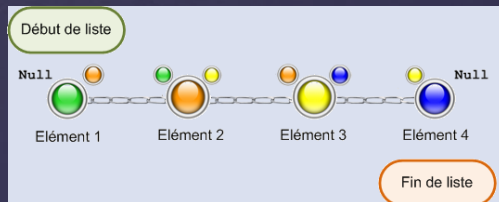
```
<terminated> TableHashage [Java Application] C:\Program Files\Java\
numéro de Alf=119034115
numéro E.T existe =false
Affiche la liste des clés=[Maison, Alf, Maman]
Maison=17314053
Alf=119034115
Maman=101990594
```

Une liste chaînée est dite récursive, car chaque élément de la liste contient :

- Une référence sur la liste,
- Une référence sur l'élément précédent,
- Une référence sur l'élément suivant.

L'utilisation de telle liste est avantageuse pour insérer des éléments en milieu de chaîne mais l'accès sera de plus en plus long, à mesure que la chaîne grandira.

Voir <https://www.irif.univ-paris-diderot.fr/~hf/verif/ens/an08-09/IF2/Cours11/C101.htm>



```
1 package fr.ifadelorozoy;
2
3 import java.util.LinkedList;
4
5
6 public class ListeChaine {
7
8     public static void main(String[] args) {
9         LinkedList<Object> l = new LinkedList<>();
10
11         l.add(12);
12         l.add("toto ! !");
13         l.add(12.20f);
14
15         for (int i = 0; i < l.size(); i++)
16             System.out.println("Élément à l'index " + i + " = " + l.get(i));
17
18         System.out.println("\n \tParcours avec un itérateur ");
19         System.out.println("-----");
20
21         ListIterator<Object> li = l.listIterator();
22
23         while (li.hasNext()){
24             if(li.hasPrevious()){
25                 System.out.println("objet précédent = " +li.previous());
26                 li.next();
27             }
28             System.out.println("objet courant = " + li.next());
29         }
30     }
31 }
32 }
```

```
<terminated> ListeChaine [Java Application] C:\P
Élément à l'index 0 = 12
Élément à l'index 1 = toto ! !
Élément à l'index 2 = 12.2

Parcours avec un itérateur

-----
objet courant =12
objet précédent =12
objet courant =toto ! !
objet précédent =toto ! !
objet courant =12.2
```

Structure type LIFO (Last In First Out), ou l'élément inséré dans la liste en dernier, est sorti en premier. Utilisation de la méthode **pop**.



```
9 import java.awt.Color;
10 import java.util.ArrayDeque;
11 import java.util.Deque;
12
13 public class PileExemple {
14
15     public static void main(String[] args) {
16         //Déclaration d'une pile de couleur
17         Deque<Color> pile = new ArrayDeque<>() ;
18         //Ajoute en haut de la pile l'anneau rouge
19         pile.push(Color.RED);
20         //Ajoute en haut de la pile l'anneau violet
21         pile.push(new Color(84,22,180));
22         //Ajoute en haut de la pile l'anneau bleu
23         pile.push(Color.BLUE);
24         //Ajoute en haut de la pile l'anneau cyan
25         pile.push(Color.CYAN);
26         //Ajoute en haut de la pile l'anneau vert
27         pile.push(Color.GREEN);
28         //Ajoute en haut de la pile l'anneau jaune
29         pile.push(Color.YELLOW);
30         //Ajoute en haut de la pile l'anneau orange
31         pile.push(Color.ORANGE);
32         //Ajoute en haut de la pile la boule rouge foncée
33         pile.push(Color.RED);
34
35         //Creation d'un panneau graphique
36         JPanel jPanel = createWindow();
37
38         //Recuperation de
39         JLabel label = null;
40         Color color = null;
41
42         // Recupere la couleur au plus haut de la pile soit rouge
43         color = pile.pop();
44         //Creation d'un label Boule de la couleur recuperee
45         label = createLabel("Boule",color);
46         jPanel.add(label);
47
48         //Ajoute les couleurs au plus de la pile tant que la liste n est pas videe
49         while(!pile.isEmpty()){
50             color = pile.pop();
51             label = createLabel("Anneau",color);
52             jPanel.add(label);
53         }
54     }
55 }
```



Structure type FIFO (First In First Out), ou l'élément inséré dans la liste en dernier, est sorti en premier. Tel que les files d'impression ou bien les files d'attente. Utilisation de la méthode **poll**.



```
1 package fr.ifadelorozoy;
2
3
4 import java.util.PriorityQueue;
5
6 public class FileExemple {
7
8     public static void main(String[] args) {
9         //Déclaration d'une pile de couleur
10        PriorityQueue<String> file = new PriorityQueue<>();
11        //Ajoute 5 clients dans la queue
12        for (int i = 1; i <= 6 ; i++) {
13            file.add("Client n°" + i);
14        }
15
16        //Ordre de traitement des clients tant que la file n est pas vide
17        //Premier arrive, premier servi
18        while(!file.isEmpty()){
19            System.out.println("Traitement du " + file.poll());
20        }
21    }
22 }
23
```

Problems Javadoc Declaration Search Console Progress History Synchronize

<terminated> FileExemple [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (23 sept. 2016 12:)

```
Traitement du Client n°1
Traitement du Client n°2
Traitement du Client n°3
Traitement du Client n°4
Traitement du Client n°5
Traitement du Client n°6
```


Les classes enveloppes

✓ *Integer, Double, Float, String*

La gestion des flux d'entrée/sortie

✓ *InputStreamReader, BufferedReader*

✓ *FileInputStream, ObjectInputStream*

✓ *FileOutputStream, ObjectOutputStream*

Les classes enveloppes

Les classe enveloppes (Wrappers) sont des classes qui encapsulent les données de type primitif. Depuis Java 5, les opérations de conversion d'un type primitif vers un objet d'une classe enveloppe, et la conversion inverse sont devenues automatique : boxing et unboxing.

Types primitifs	Classes enveloppes
boolean	Boolean
char	Char
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Sous-classes de la
classe Number

Toutes les classes ont une méthode :

- `parseX(String s) throw NumberFormatException`

Toutes les sous-classes Number possèdent les méthodes de conversion en type primitif :

- `byteValue()`,
- `shortValue()`,
- `intValue()`,
- `longValue`
- `doubleValue()`

```
/*
 * L'opération de «boxing» transforme un type primitif en une classe.
 * L'opération de «unboxing» transforme une classe en primitif.
 */
Integer i = new Integer(26); //Declaration d'un entier
i = 79 ; // boxing de int vers Integer
int j = i/2; // unboxing de Integer vers int
```

TP n° 4 : Utilisation des classes enveloppes

Grâce à la javadoc (F3 sur les classes enveloppes) et à la complétion d'Eclipse, réaliser les conversions suivantes :

Créer une classe ClasseEnveloppe :

1. Déclarer une chaîne « 12345 » (variable maChaine)
2. Convertir la chaîne en une classe Integer (variable monInteger)
3. Convertir l'entier en type double primitif (variable doubleValue)
4. Convertir le double en une classe Double (variable monDouble)
5. Affecter la valeur 50.0 primitive à monDouble (boxing)
6. Affecter monDouble à la variable doubleValue (unboxing)
7. Provoquer une erreur NumberFormatException

```
/*
 * L'opération de «boxing» transforme un type primitif en une classe.
 * L'opération de «unboxing» transforme une classe en primitif.
 */
Integer i = new Integer(26); //Declaration d'un entier
i = 79 ;           // boxing de int vers Integer
int j = i/2;       // unboxing de Integer vers int
```

TP n° 4 : Utilisation des classes enveloppes solution

```
3 public class ClasseEnveloppe {
4
5     public static void main(String[] args) {
6
7         //Declaration d une chaine
8         String maChaine = "12345";
9         try {
10             //Conversion en classe Integer
11             Integer monInteger = Integer.valueOf(maChaine);
12             System.out.println("monInteger=" + monInteger);
13
14             //Conversion en double primitif
15             double doubleValue = monInteger.doubleValue();
16             System.out.println("doubleValue=" + doubleValue);
17
18             //Conversion en classe Double
19             Double monDouble = Double.valueOf(doubleValue);
20             System.out.println("monDouble=" + monDouble);
21
22             monDouble = 50.0; // boxing de double vers Double
23             doubleValue = monDouble; // unboxing de Double vers double
24             System.out.println("doubleValue=" + doubleValue);
25
26             //Provocation d'une erreur NumberFormatException
27             Double.valueOf("Ceci n est pas un chiffre !");
28         }
29
30         catch (Exception e) {
31             e.printStackTrace();
32         }
33     }
34 }
35
36
```

Problems Javadoc Declaration Search Console Progress History Synchronize Outline

<terminated> ClasseEnveloppe [Java Application] C:\Program Files (x86)\Java\jre1.8.0_66\bin\javaw.exe (22 sept. 2016 16:43:23)

monInteger=12345

doubleValue=12345.0

monDouble=12345.0

doubleValue=50.0

[java.lang.NumberFormatException](#): For input string: "Ceci n est pas un chiffre !"

at sun.misc.FloatingDecimal.readJavaFormatString(Unknown Source)

at sun.misc.FloatingDecimal.parseDouble(Unknown Source)

at java.lang.Double.parseDouble(Unknown Source)

at java.lang.Double.valueOf(Unknown Source)

at fr.ifadelorozoy.ClasseEnveloppe.main([ClasseEnveloppe.java:27](#))

Un programme a souvent besoin d'échanger des informations :

- pour recevoir des données d'une source
- pour envoyer des données vers un destinataire.

Ces échanges aussi appelé flux (streams en anglais) ainsi que les données contenues, peuvent être de natures multiples :

- un fichier (texte, image, son...)
- une socket réseau
- un autre programme, etc ...

En Java, les flux peuvent être divisés en plusieurs catégories :

- les flux d'entrée (input stream)
- les flux de sortie (output stream)
- les flux de traitement de caractères
- les flux de traitement d'octets

Les classes de gestion des flux

Il y a de nombreuses classes, pour savoir laquelle choisir, il faut comprendre la dénomination des classes.

Le nom des classes se compose d'un préfixe et d'un suffixe.

Voir <http://www.jmdoudoux.fr/java/dej/chap-flux.htm>

Les 4 suffixes possibles :

	Flux d'octets	Flux de caractères
Flux d'entrée	InputStream	Reader
Flux de sortie	OutputStream	Writer

Exemple de préfixes possibles :

Type de traitement / Nature de la données	
Buffered	Mise en mémoire tampon
Print	Impressions formatées (ex System.out)
File	Fichier
InputStream / OutputStream	Conversion octets/caractères

Les flux d'entrée / sorties exemple

```
3 import java.io.BufferedReader;
4 import java.io.FileInputStream;
5 import java.io.InputStream;
6 import java.io.InputStreamReader;
7
8 public class FluxEntreeSortie {
9
10     public static void main(String[] args) {
11
12         // Nom d'un fichier à lire
13         String fichier = "D:\\katy\\doc\\CoursJava\\TP\\MonFichierTexte.txt";
14
15         // lecture du fichier texte
16         try {
17             // Creation d un flux d entree sur un fichier
18             InputStream ips = new FileInputStream(fichier);
19
20             // Creation d un flux de caractere d entree sur le fichier texte
21             InputStreamReader ipsr = new InputStreamReader(ips);
22
23             // Creation d'une memoire tampon sur le flux de caractere
24             BufferedReader br = new BufferedReader(ipsr);
25             String ligne;
26             System.out.println("*****Affichage du texte *****");
27             while ((ligne = br.readLine()) != null) {
28                 System.out.println(ligne);
29             }
30             br.close();
31         } catch (Exception e) {
32             System.out.println(e.toString());
33         }
34     }
35 }
```

Console

<terminated> FluxEntreeSortie [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (22 sept. 2016 23:00:24)

*****Affichage du texte *****

C'est l'heure d'aller au lit !

Juste le temps de jeter un oeil à instagram,
facebook, pinterest et regarder rapidement
l'intégrale d'une série sur Netflix.

Objectif savoir utiliser :

- La boucle **while**
- Les classes **InputStreamReader** et **BufferedReader**
- La classe List et ArrayList
- L'opérateur de décrémentation

Créer une classe ProgrammePolitique :

1. Copier le fichier LeProgramme.txt
2. Lire et afficher toutes les lignes du fichier dans l'ordre
3. Puis afficher le contenu à partir de la fin

TP n°5 : Le programme solution

```
1 package fr.ifadelorozoy;
2
3 import java.io.BufferedReader;
4 import java.io.FileInputStream;
5 import java.io.InputStream;
6 import java.io.InputStreamReader;
7 import java.util.ArrayList;
8 import java.util.List;
9
10 public class ProgrammePolitique {
11
12     public static void main(String[] args) {
13
14         String fichier = "D:\\katy\\doc\\CoursJava\\TexteLeProgramme.txt";
15
16         // lecture du fichier texte
17         try {
18             List<String> chaine = new ArrayList<String>();
19             InputStream ips = new FileInputStream(fichier);
20             InputStreamReader ipsr = new InputStreamReader(ips);
21             BufferedReader br = new BufferedReader(ipsr);
22             String ligne;
23             System.out.println("*****Affichage du texte *****");
24             while ((ligne = br.readLine()) != null) {
25                 System.out.println(ligne);
26                 chaine.add(ligne);
27             }
28             br.close();
29
30             System.out.println("*****Affichage du programme à l'envers *****");
31             for(int i=chaine.size() - 1; i >= 0 ; i--){
32                 System.out.println(chaine.get(i));
33             }
34
35         } catch (Exception e) {
36             System.out.println(e.toString());
37         }
38     }
39 }
40
41 }
42 }
```

```
<terminated> ProgrammePolitique [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (18)
*****Affichage du texte *****
Dans notre parti politique, nous accomplissons ce que nous promettons.
Seuls les imbéciles peuvent croire que
nous ne lutterons pas contre la corruption.
Parce que, il y a quelque chose de certain pour nous:
L'honnêteté et la transparence sont fondamentales pour atteindre nos idéaux.
Nous démontrons que c'est une grande stupidité de croire que
les mafias continueront à faire partie du gouvernement comme par le passé.
Nous assurons, sans l'ombre d'un doute, que
la justice sociale sera le but principal de notre mandat.
Malgré cela, il y a encore des gens stupides qui s'imaginent que
l'on puisse continuer à gouverner
avec les ruses de la vieille politique.
Quand nous assumerons le pouvoir, nous ferons tout pour que
soit mis fin aux situations privilégiées et au trafic d'influences
nous ne permettrons d'aucune façon que
nos enfants meurent de faim
nous accomplirons nos desseins même si
les réserves économiques se vident complètement
nous exercerons le pouvoir jusqu'à ce que
vous aurez compris qu'à partir de maintenant
nous sommes le parti libéral fédéral, la "nouvelle politique".
*****Affichage du programme à l'envers *****
nous sommes le parti libéral fédéral, la "nouvelle politique".
vous aurez compris qu'à partir de maintenant
nous exercerons le pouvoir jusqu'à ce que
les réserves économiques se vident complètement
nous accomplirons nos desseins même si
nos enfants meurent de faim
nous ne permettrons d'aucune façon que
soit mis fin aux situations privilégiées et au trafic d'influences
Quand nous assumerons le pouvoir, nous ferons tout pour que
avec les ruses de la vieille politique.
l'on puisse continuer à gouverner
Malgré cela, il y a encore des gens stupides qui s'imaginent que
la justice sociale sera le but principal de notre mandat.
Nous assurons, sans l'ombre d'un doute, que
les mafias continueront à faire partie du gouvernement comme par le passé.
Nous démontrons que c'est une grande stupidité de croire que
L'honnêteté et la transparence sont fondamentales pour atteindre nos idéaux.
Parce que, il y a quelque chose de certain pour nous:
nous ne lutterons pas contre la corruption.
Seuls les imbéciles peuvent croire que
Dans notre parti politique, nous accomplissons ce que nous promettons.
```

Objectifs, savoir utiliser

- La boucle **while**
- L'instruction **if**
- Les classes **InputStreamReader** et **BufferedReader**
- La classe List et ArrayList
- L'opérateur arithmétique modulo %
- L'opérateur d'incrémentement ++

Créer une classe EmployeModele:

1. Copier le fichier EmployeModele.txt
2. Lire et afficher toutes les lignes du fichier dans l'ordre
3. Puis afficher une ligne sur deux

TP n°6 L'employé modèle solution

```
1 package fr.ifadelorozoy;
2
3 import java.io.BufferedReader;
4 import java.io.FileInputStream;
5 import java.io.InputStream;
6 import java.io.InputStreamReader;
7 import java.util.ArrayList;
8 import java.util.List;
9
10 public class EmployeModele {
11
12     public static void main(String[] args) {
13         String fichier = "D:\\katy\\doc\\CoursJava\\TexteEmployeModele.txt";
14
15         // lecture du fichier texte
16         try {
17             List<String> chaine = new ArrayList<String>();
18             InputStream ips = new FileInputStream(fichier);
19             InputStreamReader ipsr = new InputStreamReader(ips);
20             BufferedReader br = new BufferedReader(ipsr);
21             String ligne;
22             System.out.println("*****Affichage du texte *****");
23             while ((ligne = br.readLine()) != null) {
24                 System.out.println(ligne);
25                 chaine.add(ligne);
26             }
27             br.close();
28
29             System.out.println("*****Affichage d'1 ligne sur 2 *****");
30             for (int i = 0; i < chaine.size(); i++) {
31                 if (i % 2 == 0) {
32                     System.out.println(chaine.get(i));
33                 }
34             }
35
36             } catch (Exception e) {
37                 System.out.println(e.toString());
38             }
39
40     }
41
42 }
```

<terminated> EmployeModele [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw

*****Affichage du texte *****

Cet employé modèle est toujours en train de travailler à son bureau avec assiduité et diligence, sans jamais perdre son temps en jasant avec ses collègues. Jamais il ne refuse de passer du temps pour aider les autres et malgré cela, il termine ses projets à temps. Très souvent, il rallonge ses heures pour terminer son travail, parfois même en sautant les pauses café. C'est une personne qui n'a absolument aucune vanité en dépit de ses accomplissements remarquables et de sa grande compétence en informatique. C'est le genre d'employé de qui on parle avec grande estime et respect, le genre de personne dont on ne peut se passer. Je crois fermement qu'il est prêt pour la promotions qu'il demande, considérant tout ce qu'il nous apporte. L'entreprise en sortira grande gagnante.

*****Affichage d'1 ligne sur 2 *****

Cet employé modèle est toujours en train de perdre son temps en jasant avec ses collègues. Jamais il ne termine ses projets à temps. Très souvent, il rallonge les pauses café. C'est une personne qui n'a absolument aucune compétence en informatique. C'est le genre d'employé de qui on peut se passer. Je crois fermement qu'il est prêt pour la porte. L'entreprise en sortira grande gagnante.